WEEK 1 · DAY 3 · OVERVIEW

Coefficients, P-Values, and Multicollinearity



Created by **Jeff Lenamon**

ABOUT 50 MIN WITH EXERCISES / 11 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Statement

BUSINESS CONTEXT

Day 2's model, 17 rating steps and duration, missed a typical unseen bond by 35.5 bp.

Before anyone quotes "a BB bond sits 164 bp over an A bond", the credit team asks how sure each coefficient is.

It also asks which columns earn a place: sector, and two more measures of a bond's length that the file carries.

The bonds and issuers are invented for this course.

OBJECTIVES

Refit Day 2's model in statsmodels and read its coefficient table.

Test sector column by column and all together, find a near-copy feature and drop it, score the choices on the test bonds, and find where the error bars themselves go wrong.

ABOUT 50 MIN WITH EXERCISES / 11 SECTIONS / 4 EXERCISES

Your Route Through Day 3

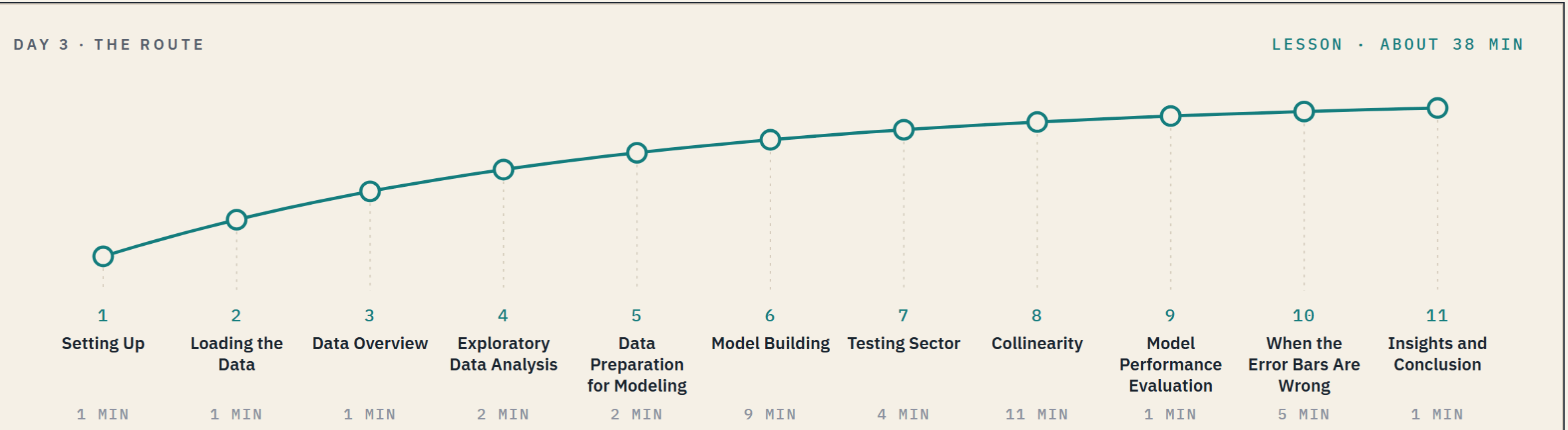


FIGURE 1 Eleven sections, in order, with about how long each takes.

Time. About 38 minutes for the lesson, then about 12 for the four exercises.

Fix the seed and load the libraries.

Setting Up

THE SETUP

```
import os
from pathlib import Path
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
SEED = 42
DATA = "../../../data/"
if not Path(DATA + "benchmark.csv").exists():
    DATA = "https://raw.githubusercontent.com/lenamonj/python-fixed-income/main/data/"
```

THE LIBRARIES

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
import statsmodels.api as sm
from sklearn.linear_model import LinearRegression
```

Read the bonds and add years to maturity.

Loading the Data

THE DATA FILE

- **`ticker`**: the issuer's short code, shared by all of its bonds
- **`sector`**: one of 10 industry sectors
- **`rating`**: credit rating, AAA to CCC
- **`maturity`** and **`as_of_date`**: the repayment date and the file's date
- **`oas`**: option-adjusted spread, bp, **the target**
- **`duration`**: modified duration, years
- **`spread_duration`**: spread duration, years
- **`coupon`** (Exercise 1) and **`dts`** (Exercise 4)

READING THE FILE, THEN ADDING YEARS TO MATURITY

```
bonds = pd.read_csv(DATA + "benchmark.csv")
ratings = pd.read_csv(DATA + "ratings.csv")
data = bonds.merge(ratings[["rating", "score", "grade"]], on="rating")
data["years_to_maturity"] = (pd.to_datetime(data["maturity"]) - pd.to_datetime(data["as_of_date"])).dt.days / 365.25
```

Data Overview

THE CALL

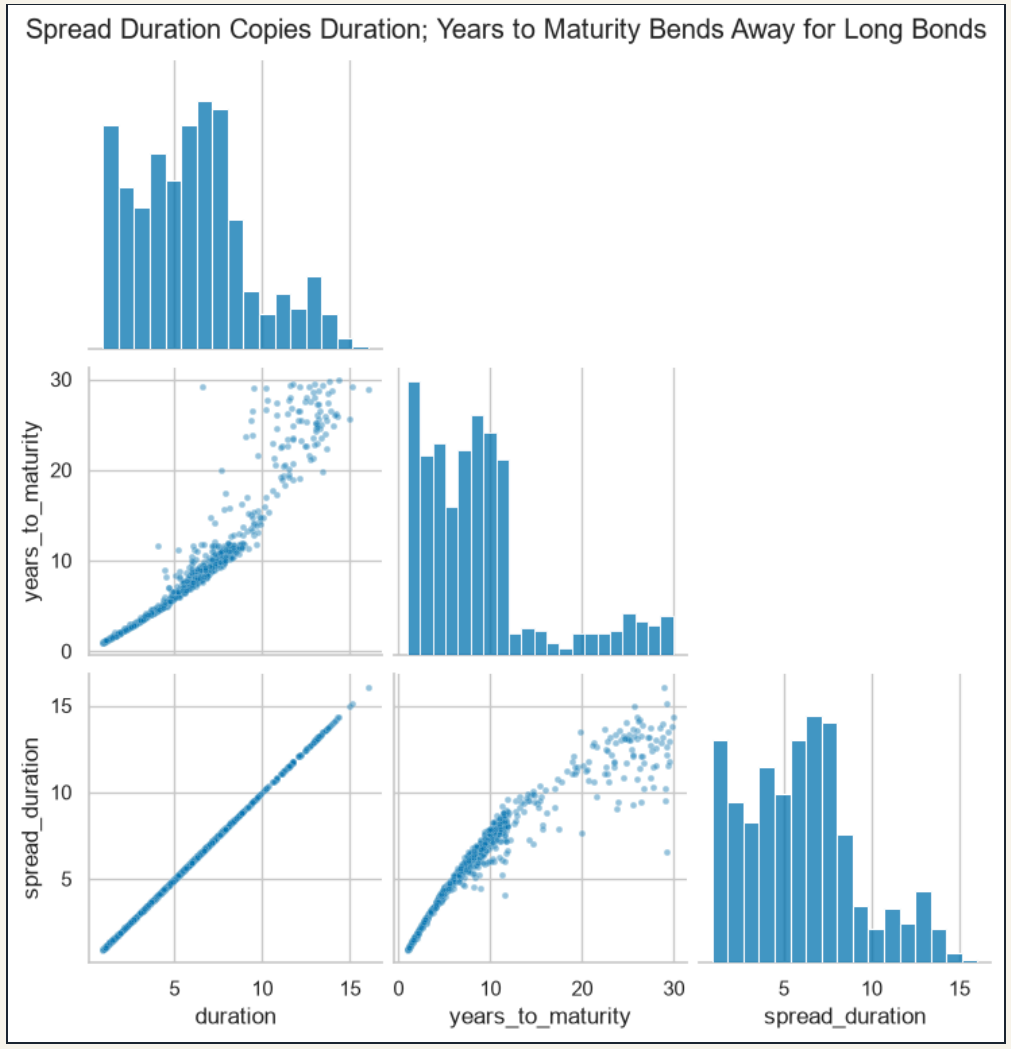
```
data[["duration", "years_to_maturity", "spread_duration"]].describe()
```

THREE COLUMNS OF LENGTH

	DURATION	YEARS_TO_MATURITY	SPREAD_DURATION
count	821.000	821.000	821.000
mean	5.981	8.910	5.981
std	3.253	6.917	3.253
min	0.915	0.964	0.915
25%	3.434	4.030	3.434

The file holds three columns that measure one bond's length, so we look at how closely they agree.

Exploratory Data Analysis



READING THE GRID

Spread duration sits on duration exactly: in this file it is a copy.

Years and duration rise together, correlation 0.942, and bend apart for long bonds: a 30-year bond's duration stops near 16.

THE CHECKS

```
Spread duration equals duration on every bond: True
Correlation of duration and years to maturity: 0.942
```

Near-copies will matter once they sit in one model. First, we ask how sure a single coefficient is.

Data Preparation for Modeling

RATING AND SECTOR COLUMNS

```
rating_order = ratings["rating"]
rating_category = pd.Categorical(data["rating"], categories=rating_order)
rating_dummies = pd.get_dummies(rating_category, prefix="rating", dtype=float).drop(columns="rating_A")
sector_dummies = pd.get_dummies(data["sector"], prefix="sector", dtype=float).drop(columns="sector_Consumer")
```

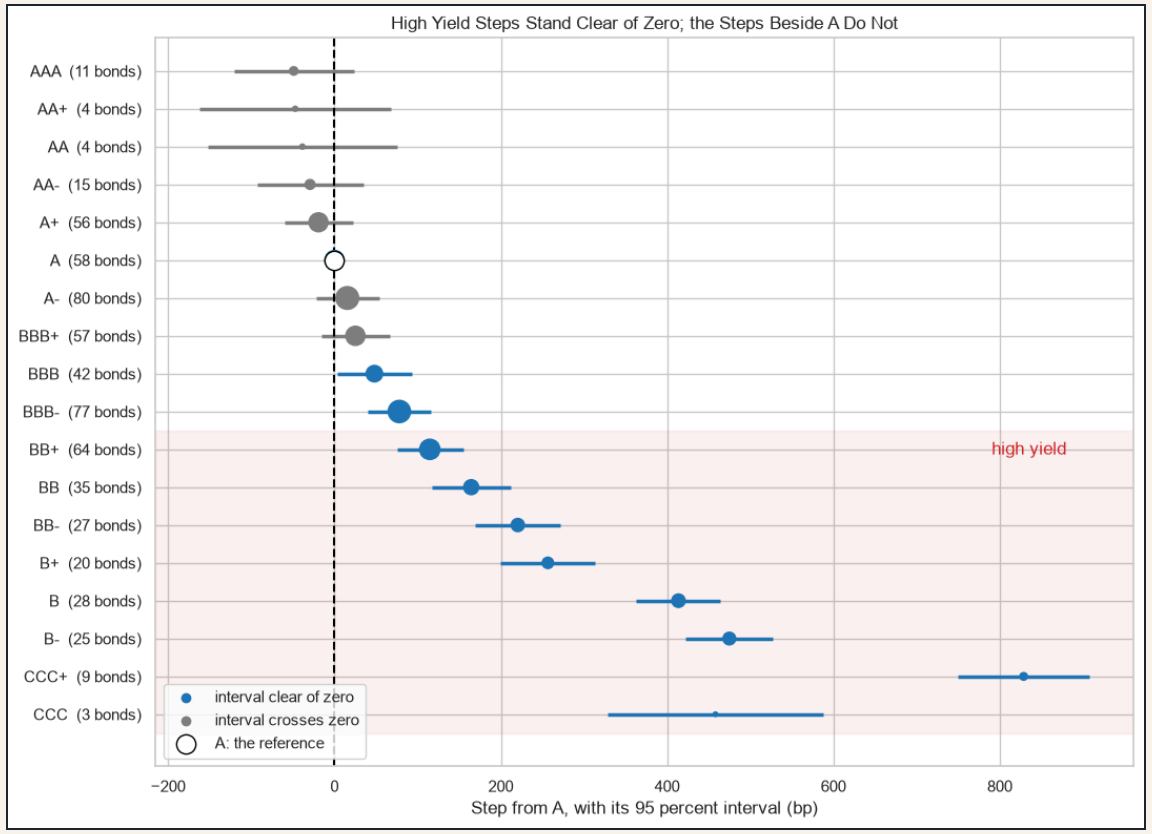
FEATURES AND THE SPLIT

```
lengths = data[["duration", "years_to_maturity", "spread_duration"]]
X = pd.concat([rating_dummies, sector_dummies, lengths], axis=1)
y = data["oas"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=SEED)
print("Training bonds:", len(X_train), " Test bonds:", len(X_test))
```

```
Training bonds: 615  Test bonds: 206
```

The same bonds, split and columns as Day 2's chosen model.

Model Building



THE COEFFICIENT TABLE

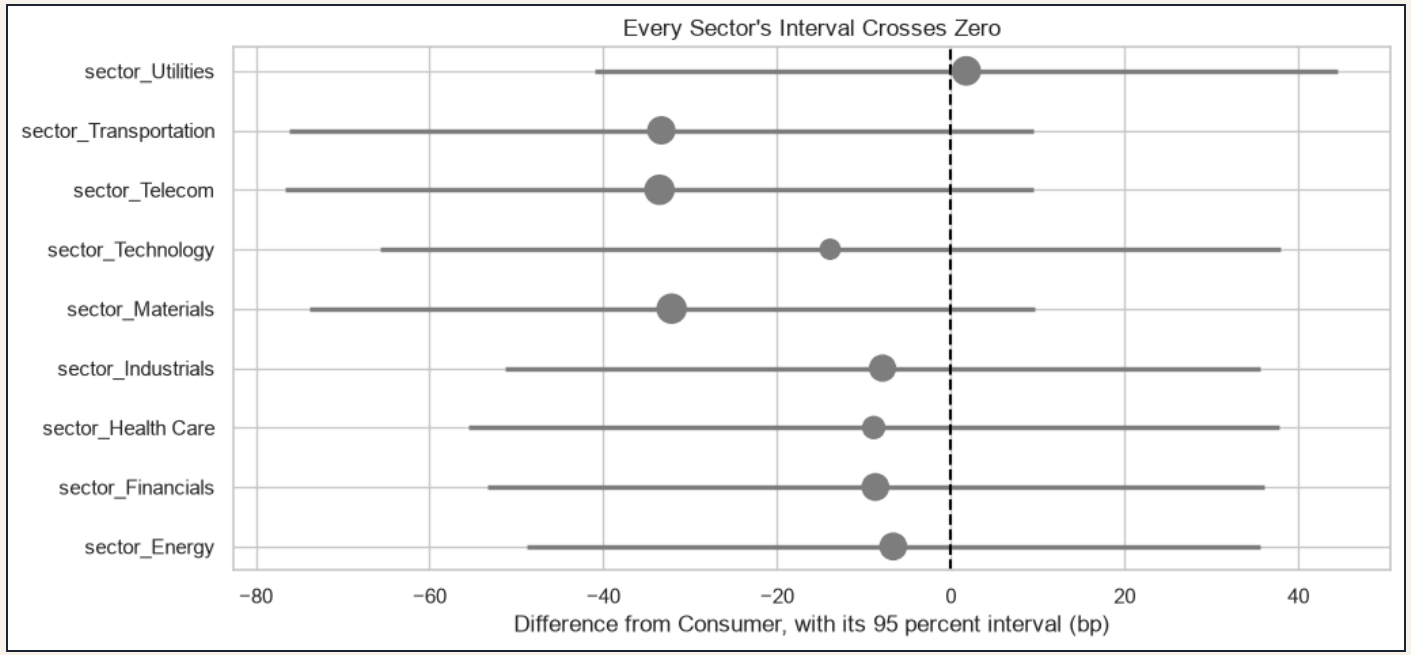
	COEF.	STD.ERR.	P>|T|
const	59.653	16.795	0.000
rating_AAA	-48.992	36.811	0.184
rating_AA+	-47.188	58.362	0.419
rating_AA	-38.617	57.870	0.505
rating_AA-	-29.421	32.437	0.365

THE FIT ON THE TRAINING BONDS

```
R-squared: 0.6727  Adjusted R-squared: 0.6628
Training bonds less coefficients: 596
```

Few bonds make a wide interval. Next, we look at a column with nothing in it.

Testing Sector



THE F-TEST

```
F: 0.82  p-value: 0.599  columns tested: 9
```

THE NINE SECTOR ROWS

	COEF.	P>|T|
sector_Energy	-6.641	0.758
sector_Financials	-8.696	0.702
sector_Health Care	-8.888	0.709
sector_Industrials	-7.869	0.723
sector_Materials	-32.146	0.131

F 0.82, p-value 0.599: the model with sector cannot be told from the model without it.

Multicollinearity

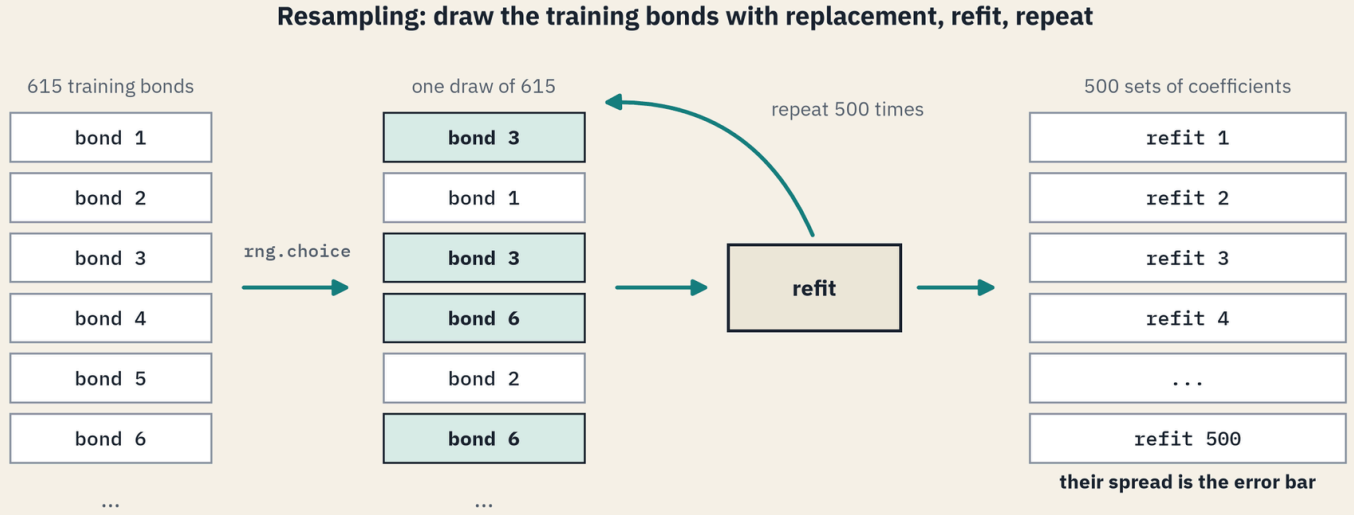


FIGURE 3 Resampling: draw 615 training bonds with replacement, refit, and repeat 500 times for 500 sets of coefficients.

THE QUESTION

Years to maturity joins duration in the model.

Duration's interval was -0.18 to 5.50 bp per year.

Sketch it on a line, then sketch where duration's interval lands with years beside it.

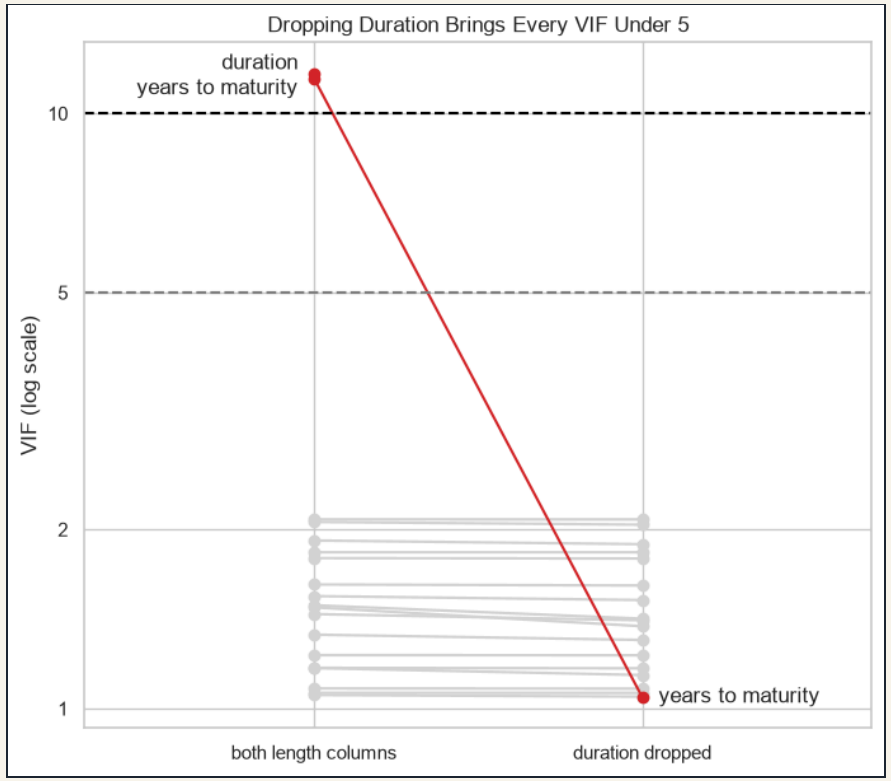
RESAMPLING

Draw 615 training bonds with replacement, refit, and repeat.

`rng.choice(615, size=615)` draws the row positions.

Resampling, also called the bootstrap, asks how much a coefficient moves on other bonds like these.

Dropping a Feature by VIF



THREE ADJUSTED R-SQUARED SCORES

```
print("Drop years, keep duration:", round(ols_model.rsquared_adj, 4))
print("Drop duration, keep years:", round(final_model.rsquared_adj, 4))
print("Keep both:                ", round(pair_model.rsquared_adj, 4))
```

```
Drop years, keep duration: 0.6628
Drop duration, keep years: 0.6648
Keep both:                 0.6669
```

EVERY VIF AGAIN

```
Years to maturity now: 1.05  Largest: 2.08
```

Dropping duration costs least, so duration goes and years to maturity stays.

Score three length choices on the test bonds.

Model Performance Evaluation

THE SCORING LOOP

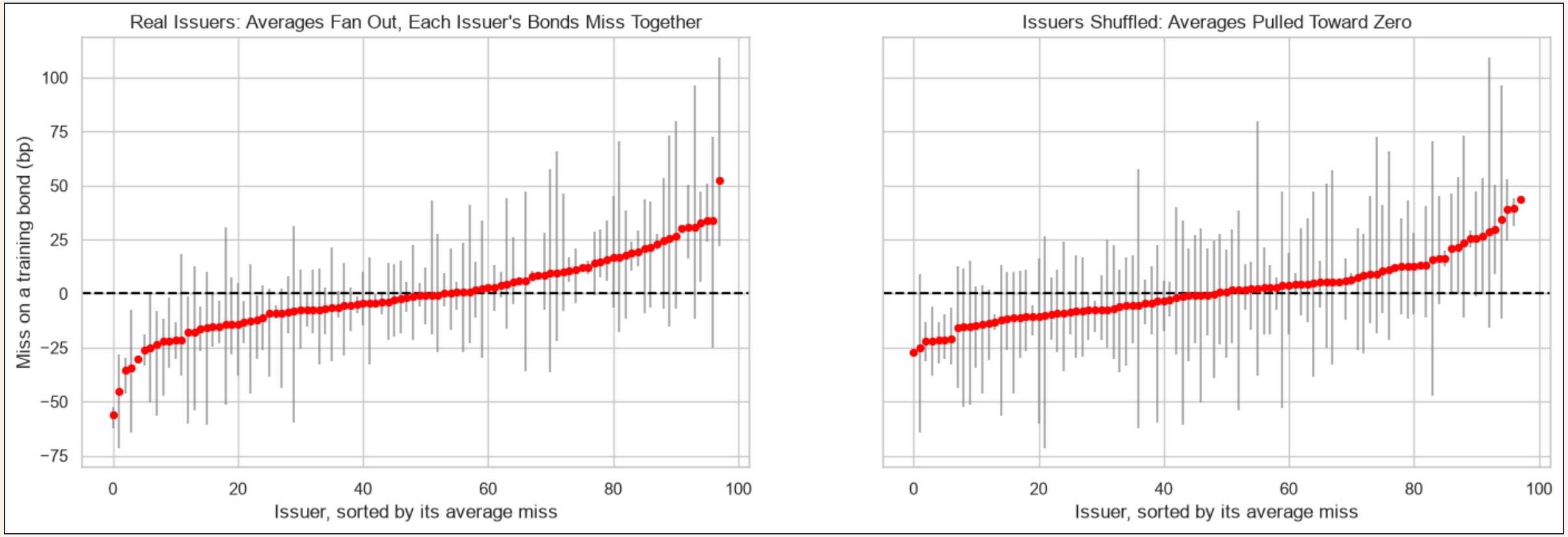
```
scores = {}
for name, model, features in [("Duration", ols_model, features_steps), ("Both", pair_model, features_pair),
                              ("Years to maturity", final_model, features_final)]:
    fitted = model.predict(sm.add_constant(X_test[features]))
    scores[name] = {"MAE (bp)": mean_absolute_error(y_test, fitted), "RMSE (bp)": np.sqrt(mean_squared_error(y_test, fitted)),
                    "R-squared": r2_score(y_test, fitted)}
```

THREE MODELS, SCORED ON THE TEST BONDS

	DURATION	BOTH	YEARS TO MATURITY
MAE (bp)	35.539	36.523	35.445
RMSE (bp)	69.135	68.839	68.812
R-squared	0.818	0.819	0.820

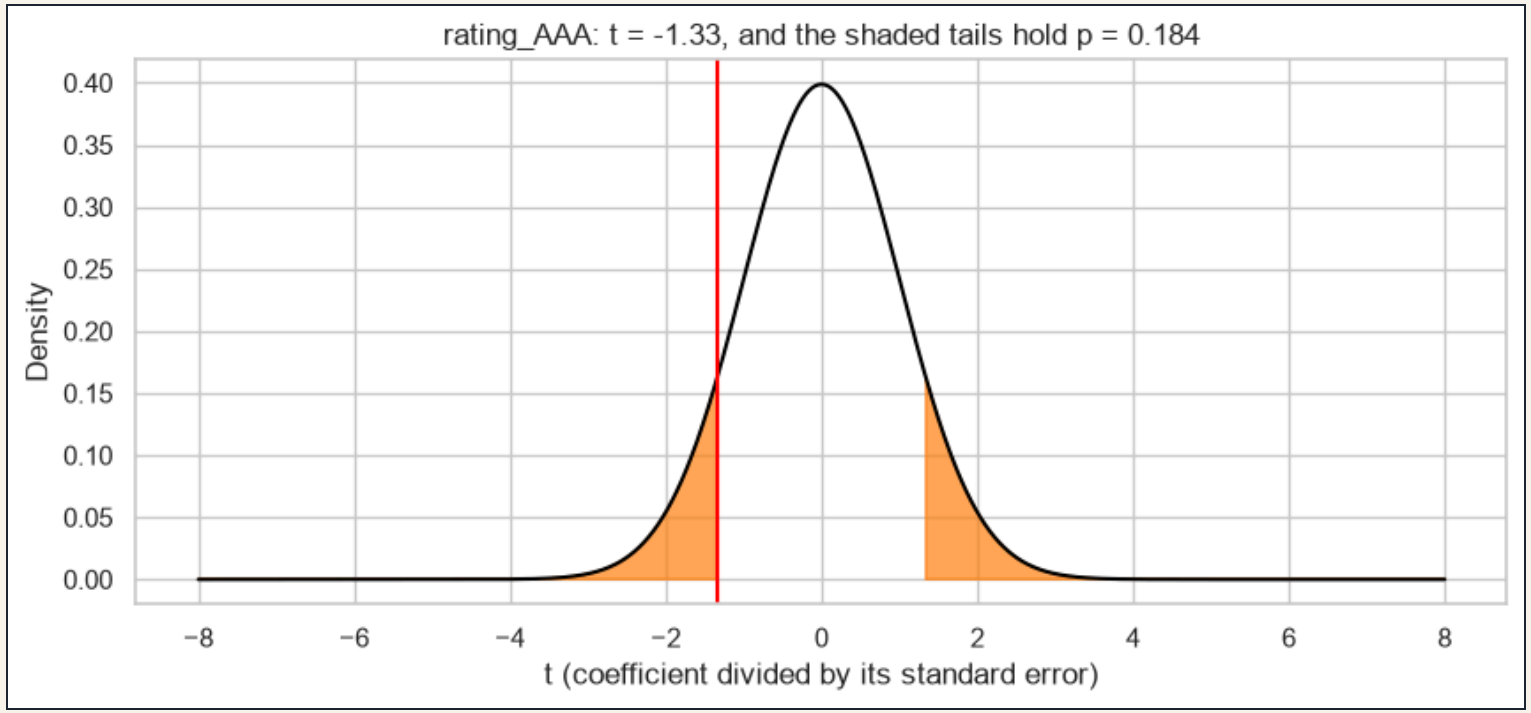
We keep the rating steps and years to maturity: the same fit, and a length coefficient that can be read.

When the Error Bars Are Wrong



Two assumptions sit under every error bar, one noise size and independent bonds, and this file breaks both.

Insights and Conclusion



THE CALL

```
draw_p_value("rating_AAA")
```

AAA's t of -1.33 leaves 0.184 of the curve in the two tails: that share is its p-value.

WHAT THE DAY FOUND

Read the error bar before the number: high yield steps stand clear of zero, and the steps beside A cannot be told from zero in bp.

An error bar is only as good as its assumptions, and in bp this file breaks two.

Day 4 checks those assumptions one by one.

You Can Now

- Fit a regression with statsmodels and read its standard errors, t values, p-values and 95 percent intervals.
- Draw a p-value as two tails of the t distribution, and coefficients as a forest plot.
- Test a group of columns together with an F-test, and say why fishing finds noise.
- Find a near-copy feature with VIF and drop it by adjusted R-squared.
- Check error bars by resampling, and name the two assumptions under them.

NEXT
WEEK 1 · DAY 4

Regression Assumptions and Log Spreads

On Day 4 you test the assumptions behind the error bars and take logs.